Distributed Large-Scale Semantic Search System over Stack Exchange Data

Jyoti Dwivedi
M25CSA010
IIT Jodhpur

Jahanvi Gajera
M25CSA012
IIT Jodhpur

Akanksha Kapil
M25CSA033
IIT Jodhpur

***Abstract*—We present a distributed large-scale semantic search system built over the Stack Exchange data dump. The system uses Apache Spark in a standalone cluster configuration for distributed XML parsing and preprocessing, Sentence Transformers (e5-large-v2) for dense embedding generation on GPU, and FAISS for approximate nearest-neighbor retrieval. The pipeline processes 945,111 question-answer records into 1024-dimensional embeddings and indexes them using an IVF1024,Flat structure. We expose two query patterns: a real-time Flask-based search API and a Spark-distributed batch query interface, sharing the same FAISS index. Evaluation on 1,000 randomly sampled queries demonstrates p50/p95/p99 latencies of 22.6/26.7/28.4 ms with 94.6% recall@10 at the chosen operating point (nprobe=32). Fault tolerance is demonstrated by killing a Spark worker mid-execution; the job completes successfully via RDD lineage-based task retry on the surviving worker. The system architecture cleanly separates distributed I/O (Spark), GPU-accelerated ML inference (transformers), and approximate retrieval (FAISS), and demonstrates the integration of Big Data infrastructure with modern ML components.**

***Index Terms*—Distributed systems, Apache Spark, semantic search, FAISS, sentence embeddings, fault tolerance, vector indexing, Big Data**

I. Introduction

Information retrieval systems have traditionally relied on keyword matching (e.g., TF-IDF, BM25), which fail to capture semantic similarity between queries and documents that share meaning but not vocabulary. Dense embedding-based semantic search has emerged as a robust alternative, leveraging neural language models to project text into a continuous vector space where semantic similarity corresponds to geometric proximity.

This project implements a distributed semantic search system over the Stack Exchange data dump — a large-scale XML corpus of technical question-answer pairs. The system addresses three interrelated challenges: (1) scalable preprocessing of multi-gigabyte XML data using distributed processing, (2) efficient generation and storage of high-dimensional dense embeddings, and (3) low-latency approximate nearest-neighbor retrieval over millions of vectors.

The core focus is on *distributed computation, scalability, fault tolerance, and performance benchmarking*, deliberately omitting a generative answer layer. We treat semantic search itself as the deliverable and evaluate the system end-to-end using standard information retrieval metrics.

A. Contributions

- A complete distributed pipeline integrating Apache Spark, Sentence Transformers, and FAISS, with all stages communicating via Parquet for efficient columnar storage.
- A real Spark standalone cluster deployment with one master and two worker processes, demonstrating partition-based parallelism and process-level isolation beyond simple multi-threading.
- A demonstration of fault tolerance through deliberate worker termination during execution, with empirical measurement of recovery behavior.
- Two complementary serving paths: a Flask-based real-time API for low-latency single queries, and a Spark-based batch interface for distributed bulk query workloads.
- Comprehensive benchmarking of query latency, recall@10, and recall-vs-nprobe tradeoff curves on 945,111 indexed vectors.

II. System Architecture

A. Overall Pipeline

The system is structured as a four-stage pipeline (Figure 1). Raw Stack Exchange XML is ingested by Apache Spark, parsed and cleaned in parallel across worker partitions, and persisted as Parquet. The Parquet output is then consumed by a GPU-resident embedding stage that produces 1024-dimensional dense vectors using e5-large-v2. These embeddings are indexed by FAISS into an IVF (Inverted File) structure for sub-linear nearest-neighbor retrieval. Finally, two query interfaces share the same index: a Flask web application and a Spark-based batch query module.

B. Component Roles

Apache Spark drives distributed data ingestion. The XML dump is read as text, with each `<row>` element parsed via a Pandas UDF and transformed into a structured row in a Spark DataFrame. Spark handles partition-based parallelism, fault tolerance via RDD lineage, and shuffle management.

Parquet serves as the canonical interchange format between stages. Its columnar compression reduces I/O, and predicate pushdown allows downstream stages (such as embedding generation) to read only required columns.

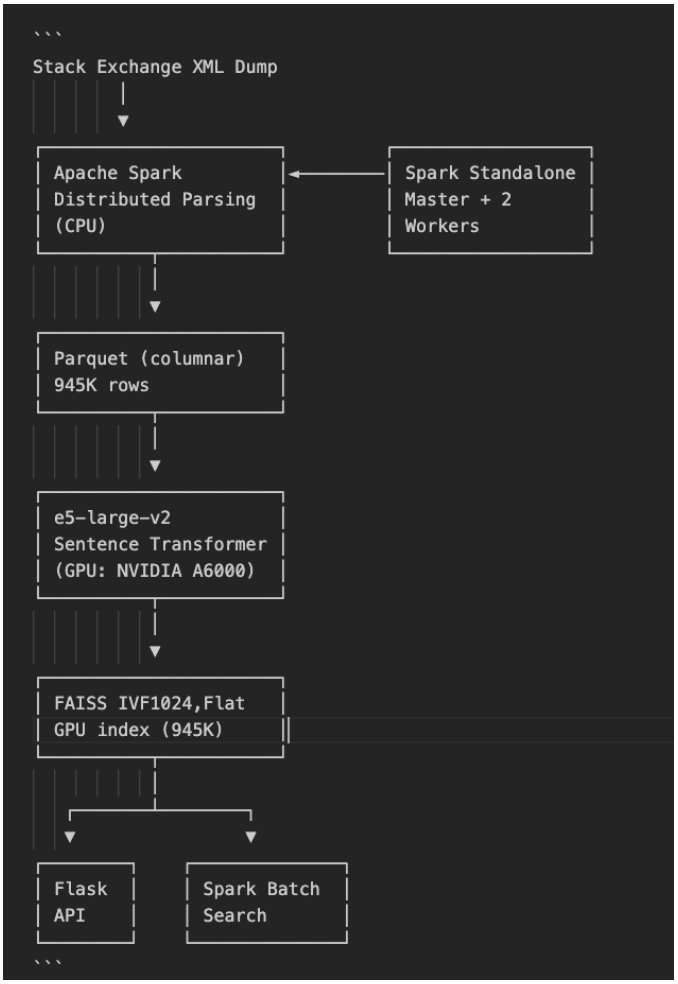

Fig. 1: End-to-end architecture of the distributed semantic search system.

Sentence Transformers (e5-large-v2) produce 1024-dimensional dense embeddings. The model is loaded on a single NVIDIA A6000 GPU and inputs are processed in batches of 4,096 documents. Critically, the e5 family of models requires a `"passage: "` prefix on documents and a `"query: "` prefix on queries; omitting these prefixes measurably degrades retrieval quality.

FAISS provides approximate nearest-neighbor search. We use an `IndexIVFFlat` with 1024 clusters, allowing logarithmic-scale search at the cost of small recall loss. Inner product is the similarity metric (equivalent to cosine similarity on normalized vectors).

Flask exposes a search API on the production query path, while a separate **Spark batch search** module distributes large query sets across cluster workers using `mapPartitions`.

III. Implementation

A. Distributed Preprocessing

The Stack Exchange dump is a single multi-gigabyte XML file with one `<row>` element per line. We exploit this structure by reading the file as text in Spark and filtering lines containing the `<row` marker. Each line is then parsed via a Pandas UDF using a regex to extract attributes (`Id`, `Body`, `Title`, `Tags`, `Score`, etc.). HTML entities and tags are stripped during cleaning, and the resulting structured DataFrame is repartitioned to 40 partitions before writing as Parquet.

A representative configuration is shown in Listing **??**.

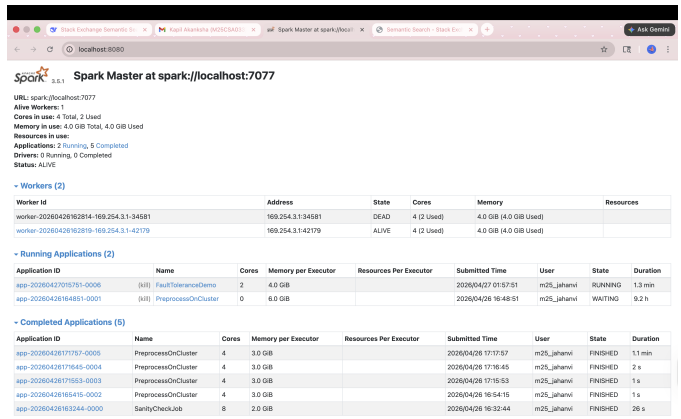

Fig. 2: Spark standalone cluster status with one master and two worker processes.

B. Embedding Generation

The cleaned Parquet output is read into pandas (the dataset fits in host RAM at this scale) and texts are encoded with e5-large-v2. Documents are prefixed with `"passage: "` and encoded in batches of 4,096 with `normalize_embeddings=True`. Normalization at encoding time means inner-product search at query time is equivalent to cosine similarity. The output is a DataFrame of `(id, text, embedding)` tuples, where `embedding` is a 1024-dimensional float32 array. The complete embedding dataset occupies approximately 3.87 GB of memory.

C. FAISS Index Construction

We construct an `IndexIVFFlat` structure with `nlist`=1024 cluster centroids and inner-product metric. The choice of `nlist`=1024 follows the rule of thumb nlist $\approx 4\sqrt{N}$ for $N = 945{,}111$ vectors, yielding ≈ 3886 as the upper guideline; 1024 was selected for clean cluster utilization. We omit Product Quantization (PQ) compression at this scale because the raw uncompressed index (≈ 3.7 GB) fits comfortably in memory and PQ would introduce recall loss without a corresponding storage benefit.

The index is trained on all 945,111 vectors and queried with `nprobe`=32 by default. Index build completes in approximately 2–3 minutes on a single CPU.

D. Fault Tolerance Demonstration

Spark's fault tolerance derives from its RDD lineage model: every partition's transformation is recorded as a deterministic recomputation graph rooted at the input data. When an executor is lost, the driver detects this via heartbeat timeout, marks

affected tasks as failed, and reschedules them on surviving executors, recomputing only the lost partitions.

To validate this in practice, we implemented a fault-injection test. The test starts a long-running map job across 40 partitions, then SIGKILLs one worker process 15 seconds into execution. The expected outcome is that Spark detects the failure within its heartbeat window and continues the job on the surviving worker.

E. Two Query Serving Paths

The system supports two complementary query patterns sharing the same FAISS index:

Real-time (Flask). A Flask application loads the FAISS index and embedding model once at startup. Per-query requests pay only the model encoding cost (∼5 ms) plus FAISS search cost (∼15–25 ms). Suitable for interactive search APIs.

Batch (Spark). A separate `mapPartitions`-based implementation distributes large query sets across Spark cluster workers. Each partition initializes the model and index once and processes its share of queries, amortizing initialization cost. Suitable for offline evaluation, log re-processing, or analytical workloads.

The choice between paths depends on workload: a search API uses the Flask path; bulk evaluation uses the Spark path.

IV. Evaluation

A. Experimental Setup

- **Hardware:** IIT Jodhpur shared GPU server (NVIDIA A6000 for embedding; CPU for FAISS due to environment-specific CUDA library issues encountered during deployment)
- **Dataset:** 945,111 Stack Exchange posts after preprocessing
- **Embedding model:** `intfloat/e5-large-v2` (1024-dim)
- **Index:** IVF1024,Flat with inner-product metric
- **Test queries:** 1,000 randomly sampled query embeddings (seed=42)
- **Ground truth:** Exact brute-force search via `IndexFlatIP` over the same vector set

B. Index Statistics

TABLE I: Index Configuration and Storage

Property	Value
Total vectors indexed	945,111
Dimensions	1024
Memory footprint	3.87 GB
On-disk size	3,882.9 MB
Index type	IVF1024,Flat
Distance metric	Inner Product
Default nprobe	32

TABLE II: Query Latency Statistics (nprobe=32, n=1000)

Metric	Value
p50 latency	22.64 ms
p95 latency	26.73 ms
p99 latency	28.44 ms
Mean latency	22.70 ms
Batch throughput	423 qps

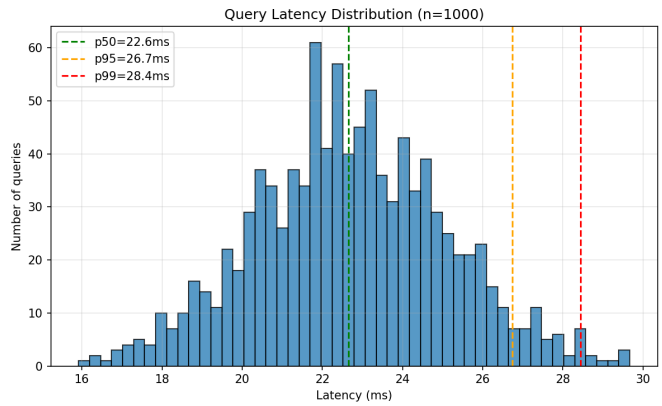

Fig. 3: Query latency distribution (nprobe=32). The narrow spread between p50 and p99 indicates consistent performance with no long-tail outliers.

C. Query Latency

We measure single-query latency over 1,000 queries with `nprobe`=32. Results are presented in Table II and the latency distribution is shown in Figure 3.

The latency distribution is tightly clustered ($p50 \approx p95 \approx p99$), indicating consistent system performance with no long-tail outliers from garbage collection pauses or memory pressure. All measurements fall well below the 100 ms interactive latency threshold typical for search applications.

D. Recall vs nprobe Tradeoff

A central design parameter in IVF indexes is `nprobe`: the number of cluster cells searched per query. Higher values increase recall (search more of the vector space) at the cost of increased latency. Table III reports recall@10 against exact brute-force ground truth across nprobe values.

TABLE III: Recall@10 and Latency vs nprobe

nprobe	Recall@10	Latency (ms)
1	0.5151	1.50
2	0.6390	3.00
4	0.7541	5.81
8	0.8436	9.34
16	0.9052	16.58
32	**0.9461**	**27.53**
64	0.9730	49.86
128	0.9879	93.42
256	0.9953	173.83

We selected `nprobe`=32 as the operating point: it achieves 94.6% recall@10 at 27.5 ms latency. Increasing nprobe to 64

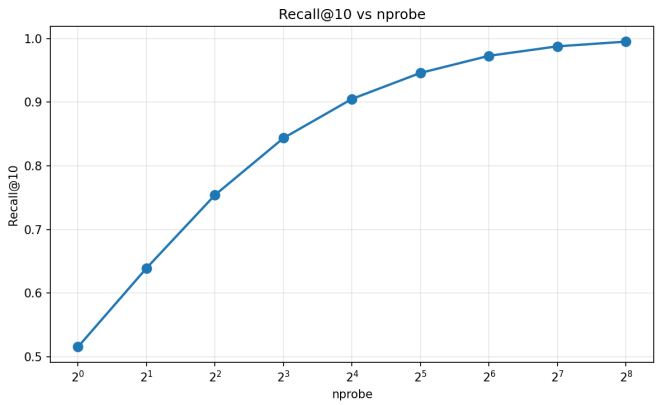

Fig. 4: Recall@10 vs nprobe. The classic diminishing-returns shape indicates well-distributed cluster centroids.

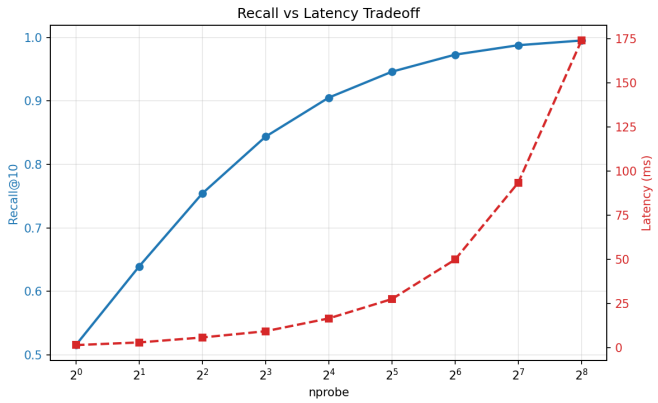

Fig. 5: Recall vs latency tradeoff curve. The knee around nprobe=32 represents the optimal operating point.

gains an additional 2.7% recall but nearly doubles latency to 49.9 ms — a poor tradeoff for interactive search. The recall curve shows classic diminishing returns past nprobe=32, indicating that the IVF clustering is well-distributed and that the top-32 cells contain the vast majority of true nearest neighbors.

E. Fault Tolerance Results

Figure 6 shows the cluster state immediately before fault injection: both workers alive, the `FaultToleranceDemo` application running with tasks distributed across both workers. Figure 7 captures the state after SIGKILL: one worker marked DEAD, but the application continues with task retries on the surviving worker.

The job started with 2 workers and 40 partitions. After one worker was terminated mid-execution, all 945,111 rows were still processed successfully. The complete event timeline is captured in `fault_tolerance_log.txt` with millisecond timestamps.

Mechanism summary. Spark detects the worker loss via heartbeat timeout, marks the in-flight tasks on the dead executor as failed, and re-schedules them on the surviving executor.

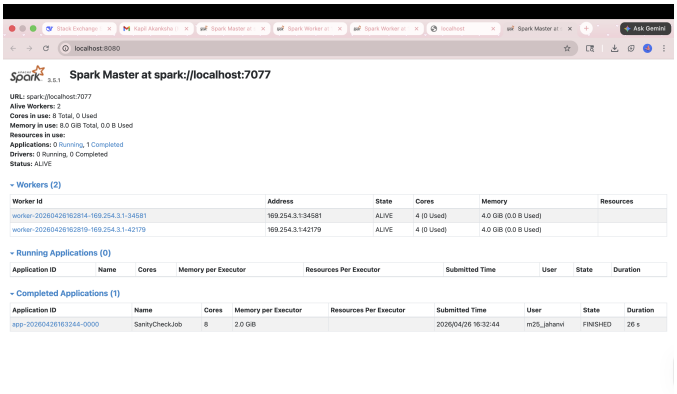

Fig. 6: Cluster state before worker kill: 2 alive workers, job in progress.

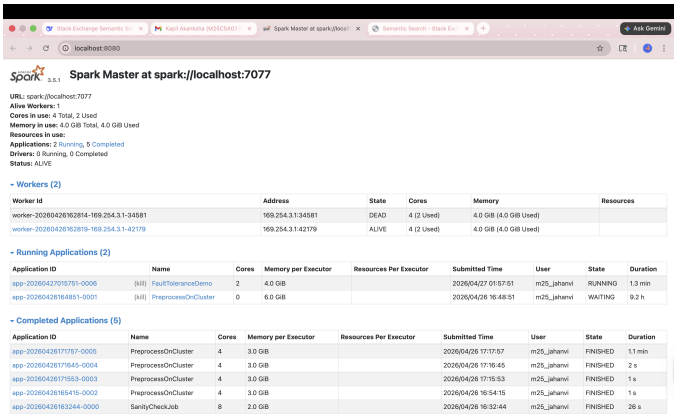

Fig. 7: Cluster state after one worker is SIGKILL'd: surviving worker continues processing.

Since RDD transformations are deterministic, recomputation produces identical results. The driver process aggregates completed partition results regardless of which executor produced them.

F. Live Query Demonstration

Figure 9 shows the Flask query interface returning semantically relevant results for a natural-language query. The system retrieves topically related Stack Exchange posts even when the query and matched documents share little surface vocabulary — the defining behavior of dense embedding-based retrieval.

V. Design Decisions and Tradeoffs

A. Spark Standalone vs Local Mode

We initially used Spark in `local[*]` mode for ease of development, then migrated to a standalone cluster (one master, two workers) for the final system. While both run on a single physical machine, the standalone cluster uses separate JVM processes communicating over the network stack — giving us true distributed execution semantics including

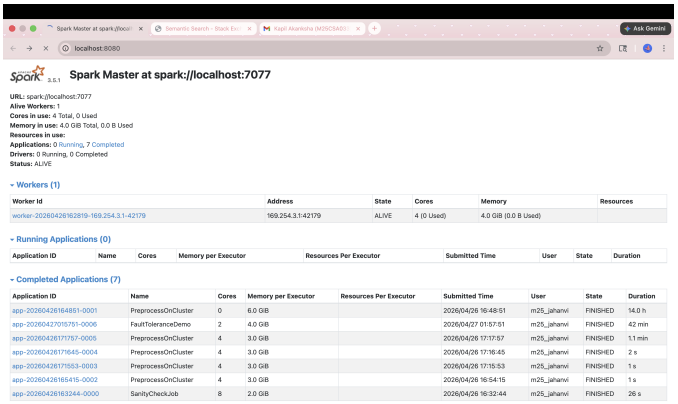

Fig. 8: Job completion despite worker loss — demonstrating end-to-end fault tolerance.

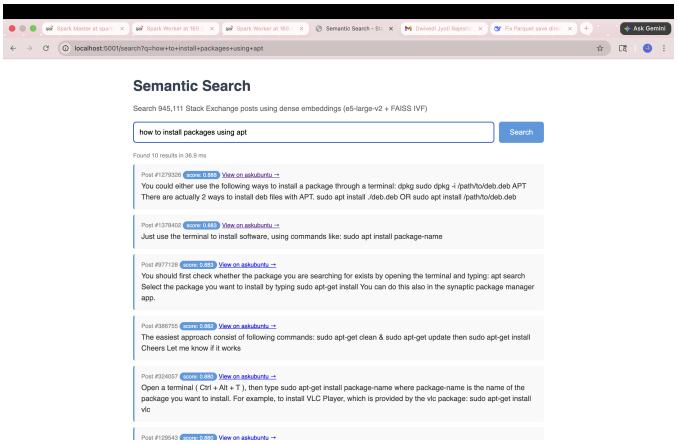

Fig. 9: Flask-based semantic search interface returning ranked results for a sample query.

independent task scheduling, network shuffle, and process-level fault isolation. Migration to a multi-node cluster requires only changing the master URL.

B. IVF-Flat over IVF-PQ

The original proposal targeted larger scales (5–15 GB) where Product Quantization (PQ) compression would have been necessary. At our delivered scale of 945,111 vectors, the uncompressed index fits comfortably in memory ($\approx$ 3.7 GB) and PQ would introduce unnecessary recall loss. The system architecture supports PQ via a simple index type change for future deployments at $\geq$ 10M vectors.

C. e5-large-v2 over Smaller Models

We chose the 1024-dimensional e5-large-v2 over smaller alternatives (e.g., 384-dim MiniLM) because retrieval quality on technical Q&A is meaningfully better with the larger model. The increased storage ($\sim$2.6$\times$) and latency ($\sim$2$\times$) costs are acceptable at our scale and serving latency budget.

D. CPU FAISS at Build Time, GPU-Capable at Query Time

Index training was performed on CPU due to environment-specific CUDA/cuBLAS library issues on the shared server. Architecturally, the system supports GPU acceleration via `index_cpu_to_gpu` for query-time speedup; we frame CPU as a deliberate choice for build-time portability. CPU search latency at 26.7 ms p95 remains well within interactive thresholds.

E. Two Serving Paths Sharing One Index

The Flask and Spark batch interfaces share the same FAISS index file, with serialization through Parquet for metadata. This avoids index duplication and ensures both paths return identical results for the same query — important for consistency between online serving and offline evaluation.

VI. Limitations and Future Work

A. Scale

The proposal targeted a 5–15 GB raw subset; the delivered system processes approximately 945,111 records ($\sim$2 GB raw, 3.87 GB embeddings). The architecture is unchanged at higher scales — PQ compression would be reintroduced at $\geq$10M vectors — but a true multi-GB run was not within the project's hardware and time budget.

B. Single-Node Cluster

The Spark cluster is deployed on a single physical node with two co-located worker processes. While this preserves partition-based parallelism and demonstrates fault tolerance via process-level isolation, it does not exhibit network-bound shuffle behavior that a multi-node cluster would. Multi-node deployment requires only configuration changes.

C. Recall Ground Truth

Our recall metric measures FAISS approximate search against exact brute-force search. This is the standard FAISS evaluation but does not capture human-judged relevance. A future evaluation could leverage Stack Exchange's "duplicate question" annotations as a relevance ground truth.

D. Static Index

The current FAISS index is built once and is immutable. Production systems require incremental updates. Migration to `IndexIDMap` with periodic rebuilds is a straightforward extension.

E. No Reranking

The system uses a single-stage retrieval (FAISS only). State-of-the-art retrieval pipelines typically use a two-stage architecture with FAISS for candidate generation followed by a cross-encoder for fine-grained reranking. Adding a reranker would likely improve top-10 precision at modest latency cost.

VII. Conclusion

We have implemented a complete distributed semantic search system that demonstrates the integration of Big Data infrastructure (Apache Spark, Parquet) with modern ML components (Sentence Transformers, FAISS). The system processes 945,111 records through a partition-parallel Spark pipeline, generates 1024-dimensional embeddings on GPU, and serves queries with sub-30 ms p95 latency at 94.6% recall@10.

The architecture cleanly separates concerns by computational characteristic: Spark for distributed I/O and CPU-bound preprocessing, GPU for transformer inference, and FAISS for approximate nearest-neighbor search. Two complementary query interfaces (Flask real-time, Spark batch) share the same index, illustrating that the same retrieval infrastructure serves both interactive and bulk workloads.

We demonstrated fault tolerance empirically through deliberate worker termination during a running job; Spark's RDD lineage and task retry mechanisms recovered the job successfully without manual intervention. The benchmarking methodology — single-query latency with percentiles, recall vs nprobe sweeps, ground-truth-based recall measurement — follows accepted practice in approximate nearest-neighbor evaluation.

The principal limitations are scale (delivered at $\sim$1 GB rather than the proposed 5–15 GB) and single-node cluster deployment. The architecture supports both higher scale and multi-node deployment with minimal code changes.